Toor — Dockerized Application Deployment on AWSEC2

Containerization, Docker Compose, Networking & CloudDeployment

---

Course Program: DESC Free IT Training of Cloud Computing

by SNSKIES

Class Name: Cloud Computing (Morning)

Assignment Report: 2nd (Week 3nd)

Instructor: Muhammad Rafid Imran

Date of Submission: May 1, 2026

Table of Contents

Toor — Dockerized Application Deployment on AWS EC2

Introduction

From a local Toor application to a cloud-hosted Docker deployment

Purpose

This report documents the practical deployment of the Toor e-commerce application using Docker and Amazon EC2. The goal is to show the complete path from the application structure and local execution to container images, Docker Compose, image verification and cloud deployment.

What the screenshots demonstrate

The evidence shows the Toor project structure, Docker build activity, containerized services, troubleshooting output, Docker image verification, an AWS EC2 instance, SSH access, image pulling, container execution and the final application running from the cloud host.

Learning outcome

The work connects several core Cloud and DevOps ideas in one small project: reproducible application environments, container networking, troubleshooting through logs, image distribution and deployment on an EC2 virtual machine.

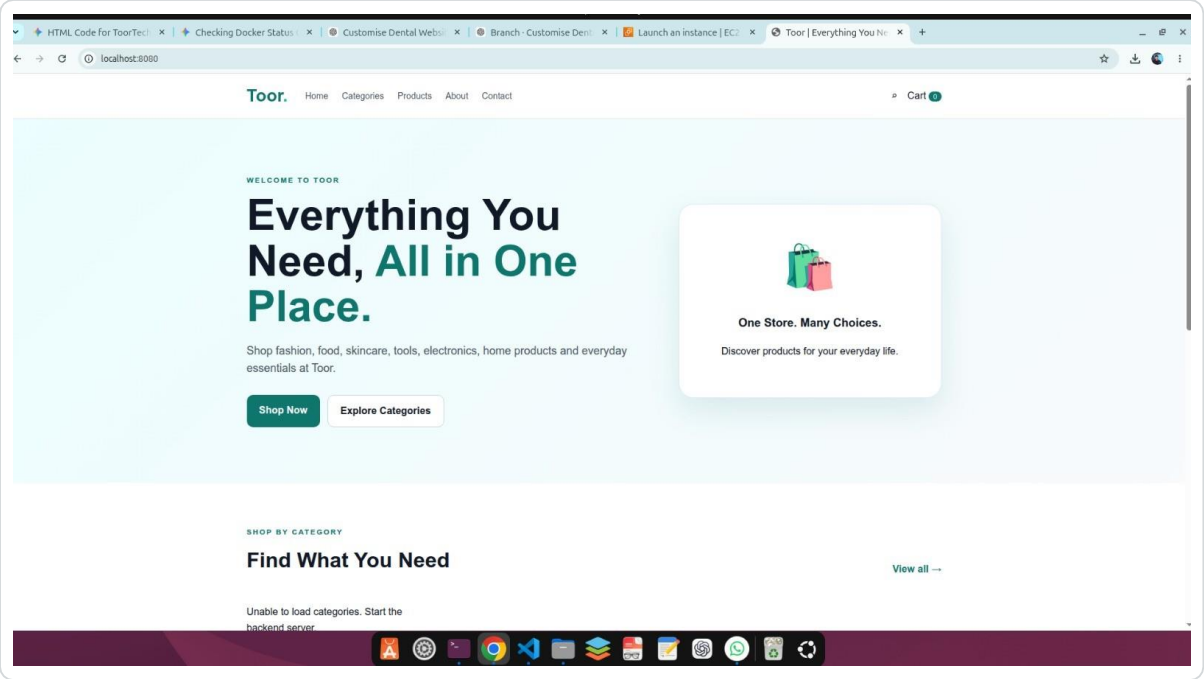
Task 1: Toor Project & Docker

Starting with the application before containerization

```
toor@toor:~/Projects/Toor_E-commerce_web$ tree -L 2 -I "node_modules|.git|build|dist|.next"
.
├── backend
│   ├── config
│   ├── controllers
│   ├── Dockerfile
│   ├── models
│   ├── package.json
│   ├── README.md
│   ├── routes
│   └── server.js
├── database
│   ├── README.md
│   ├── scripts
│   └── seed
├── frontend
│   ├── about.html
│   ├── assets
│   ├── cart.html
│   ├── categories.html
│   ├── contact.html
│   ├── css
│   ├── Dockerfile
│   ├── index.html
│   ├── js
│   ├── product.html
│   ├── products.html
│   └── README.md
├── package.json
└── package-lock.json

13 directories, 16 files
toor@toor:~/Projects/Toor_E-commerce_web$
```

Project Structure



Local Application

Q.2: Why is Docker useful for Toor?

Understanding the reason for containerization

Consistent environment

Docker packages the application together with the runtime environment and dependencies required to run it. This reduces differences between development, testing, and deployment environments.

Repeatable deployment

Instead of manually preparing every machine, the same image can be used to start the application again. This makes the deployment process easier to reproduce and troubleshoot.

Isolation

The Toor application runs inside containers rather than directly depending on the host operating system. Separate services can therefore have their own runtime environments while communicating through Docker networking.

DevOps connection

For this project, Docker is the bridge between application development and cloud deployment: build an image locally, verify it, distribute it, then run the same image on EC2.

Task 2: Dockerfile & Image Build

Turning application source into runnable images

```
toor@toor:~/Projects/Toor_E-commerce_web$ docker compose up
[+] up 15/15
 ✔ Image mongo:8                              Pulled
 ✔ Volume toor_e-commerce_web_mongodb_data    Created
 ✔ Container toor-mongodb                     Created
 ✔ Container toor-backend                     Created
 ✔ Container toor-frontend                    Created
Attaching to toor-backend, toor-frontend, toor-mongodb
toor-mongodb  | {"t":{"$date":"2026-09-02T18:20:01.090+00:00"},"s":"F",  "c":"CONTROL",  "id":12257600,"ctx":"main","msg":"MongoDB cannot start: Lin
has a known incompatibility with this version of MongoDB. See https://jira.mongodb.org/browse/SERVER-121912 for more information."}
toor-mongodb exited with code 1
toor-frontend  | /docker-entrypoint.sh: /docker-entrypoint.d/ is not empty, will attempt to perform configuration
toor-frontend  | /docker-entrypoint.sh: Looking for shell scripts in /docker-entrypoint.d/
toor-frontend  | /docker-entrypoint.sh: Launching /docker-entrypoint.d/10-listen-on-ipv6-by-default.sh
toor-frontend  | 10-listen-on-ipv6-by-default.sh: info: Getting the checksum of /etc/nginx/conf.d/default.conf
toor-frontend  | 10-listen-on-ipv6-by-default.sh: info: Enabled listen on IPv6 in /etc/nginx/conf.d/default.conf
toor-frontend  | /docker-entrypoint.sh: Sourcing /docker-entrypoint.d/15-local-resolvers.envsh
toor-frontend  | /docker-entrypoint.sh: Launching /docker-entrypoint.d/20-envsubst-on-templates.sh
toor-frontend  | /docker-entrypoint.sh: Launching /docker-entrypoint.d/30-tune-worker-processes.sh
toor-frontend  | /docker-entrypoint.sh: Configuration complete; ready for start up
toor-frontend  | 2026/09/02 18:20:01 [notice] 1#1: using the "epoll" event method
toor-frontend  | 2026/09/02 18:20:01 [notice] 1#1: nginx/1.31.4
toor-frontend  | 2026/09/02 18:20:01 [notice] 1#1: built by gcc 15.2.0 (Alpine 15.2.0)
toor-frontend  | 2026/09/02 18:20:01 [notice] 1#1: OS: Linux 7.0.0-30-generic
toor-frontend  | 2026/09/02 18:20:01 [notice] 1#1: getrlimit(RLIMIT_NOFILE): 1024:524288
toor-frontend  | 2026/09/02 18:20:01 [notice] 1#1: start worker processes
toor-frontend  | 2026/09/02 18:20:01 [notice] 1#1: start worker process 30
toor-frontend  | 2026/09/02 18:20:01 [notice] 1#1: start worker process 31
toor-frontend  | 2026/09/02 18:20:01 [notice] 1#1: start worker process 32
toor-frontend  | 2026/09/02 18:20:01 [notice] 1#1: start worker process 33
toor-frontend  | 2026/09/02 18:20:01 [notice] 1#1: start worker process 34
toor-frontend  | 2026/09/02 18:20:01 [notice] 1#1: start worker process 35
toor-backend   |
toor-backend   | > toor-backend@1.0.0 start
toor-backend   | > node server.js
toor-backend   |
toor-backend   | Database connection failed: getaddrinfo EAI_AGAIN mongodb
toor-backend exited with code 1

w Enable Watch   d Detach
```

Docker Build

```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS

toor@toor:~/Projects/Toor_E-commerce_web$ docker compose build
[+] Building 36.5s (20/20) FINISHED
 => [internal] load local bake definitions
 => => reading from stdin 1.05kB
 => [backend internal] load build definition from Dockerfile
 => => transferring dockerfile: 156B
 => [frontend internal] load build definition from Dockerfile
 => => transferring dockerfile: 96B
 => [frontend internal] load metadata for docker.io/library/nginx:alpine
 => [backend internal] load metadata for docker.io/library/node:22-alpine
 => [backend internal] load .dockerignore
 => => transferring context: 97B
 => [backend 1/5] FROM docker.io/library/node:22-alpine@sha256:c610fcdfb1d5b4740dd70c284ed3cb16bb857e0f7166196e36a5501df7a3aa32
 => => resolve docker.io/library/node:22-alpine@sha256:c610fcdfb1d5b4740dd70c284ed3cb16bb857e0f7166196e36a5501df7a3aa32
 => [backend internal] load build context
 => => transferring context: 6.59kB
 => CACHED [backend 2/5] WORKDIR /app
 => [backend 3/5] COPY package*.json ./
 => [backend 4/5] RUN npm install
 => [frontend internal] load .dockerignore
 => => transferring context: 120B
 => [frontend internal] load build context
 => => transferring context: 28.36kB
 => [frontend 1/2] FROM docker.io/library/nginx:alpine@sha256:db35bfc6b2951e7f8a72db5db120288c127ffaeeb4a6d4b95a26fead017d5913
 => => resolve docker.io/library/nginx:alpine@sha256:db35bfc6b2951e7f8a72db5db120288c127ffaeeb4a6d4b95a26fead017d5913
 => => sha256:fb597529c91648bab2279939e01b745210875081852c73000aed882c000ff5de 20.50MB / 20.50MB
 => => sha256:da4dea1b00afb53e163ae330a511ecd6978371d99794c80f439b83b609eb9d66 1.40kB / 1.40kB
 => => sha256:8cdfe4f237782ae36780c4634b1457e8d6efa6f929fc3e46b0a2074ba54acd56 1.21kB / 1.21kB
 => => sha256:0d62d88506ba127db40e7c137e3a2dcf65e563709676828820b8c82277a6caeb 404B / 404B
 => => sha256:0ea93572787841bba2f3921ece8dc2db925c22232a1388c766881ebe3bd3f67d 628B / 628B
 => => sha256:1ed8e39a7434c8975957bb78530f191c92eecbd9c7309966192ba276a7c0ac61 955B / 955B
 => => sha256:d94291c262619626bc7a27b61b090d85365e85c61f92ca49b190a26930121299 1.90MB / 1.90MB
 => => extracting sha256:d94291c262619626bc7a27b61b090d85365e85c61f92ca49b190a26930121299
 => => extracting sha256:0ea93572787841bba2f3921ece8dc2db925c22232a1388c766881ebe3bd3f67d
 => => extracting sha256:1ed8e39a7434c8975957bb78530f191c92eecbd9c7309966192ba276a7c0ac61
 => => extracting sha256:0d62d88506ba127db40e7c137e3a2dcf65e563709676828820b8c82277a6caeb
 => => extracting sha256:8cdfe4f237782ae36780c4634b1457e8d6efa6f929fc3e46b0a2074ba54acd56
```

Build / Compose Output

Q.2: What happens during docker build?

Reading the build process step by step

1. Read the Dockerfile

Docker uses the Dockerfile as the build instructions. The file defines the base image, working directory, files to copy, dependencies to install, exposed port, and startup command.

2. Create image layers

Each Dockerfile instruction contributes to the image build process. Docker can reuse cached layers when the inputs have not changed, which can make later builds faster.

3. Produce a runnable image

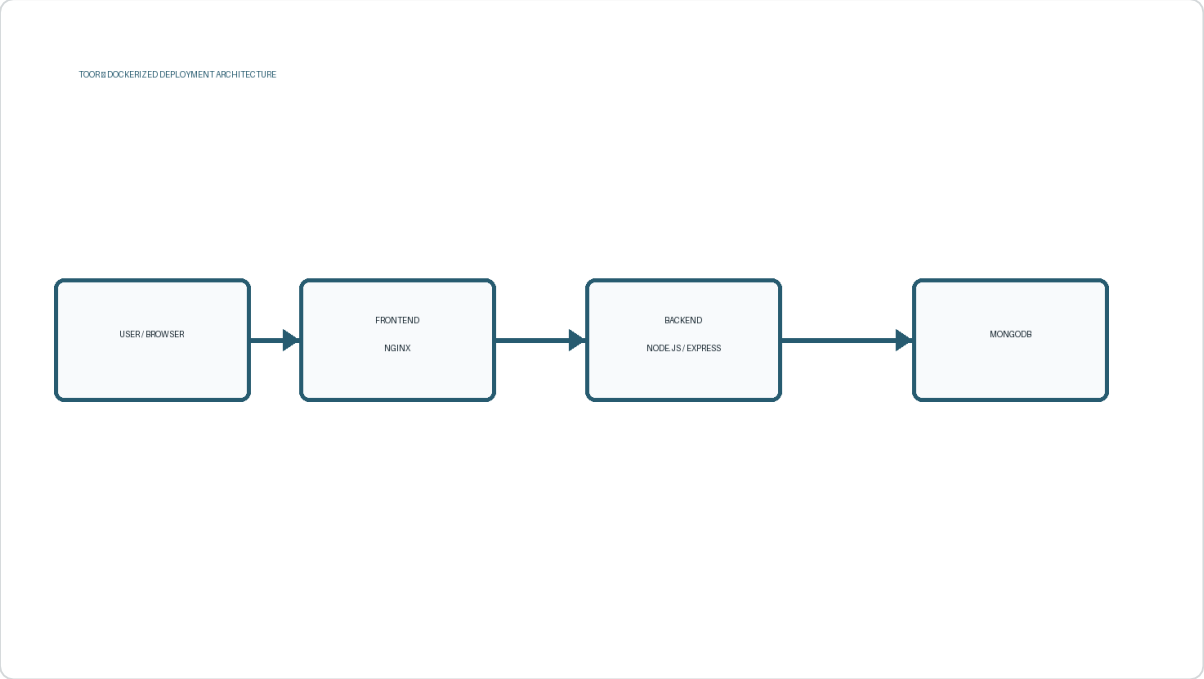
The result is an image containing what the service needs to start. The screenshots show Docker processing the frontend and backend build contexts and completing the image-building workflow.

4. Start containers

The image itself is not the running application. A container is created from the image. Docker Compose can create the required service containers together.

Task 3: Docker Compose & Networking

Connecting the Toor application services



Deployment Architecture

Containerized Images

Q.2: Why is Docker networking important?

Allowing containers to communicate by service

Application flow

The Toor architecture is represented as: User/Browser → Frontend → Backend → MongoDB. Each component has a different responsibility, while networking allows the services to communicate.

Container-to-container communication

When services share a Docker network, they can communicate without treating each container as an isolated machine. This is especially important for a backend that must reach the database service.

Important troubleshooting idea

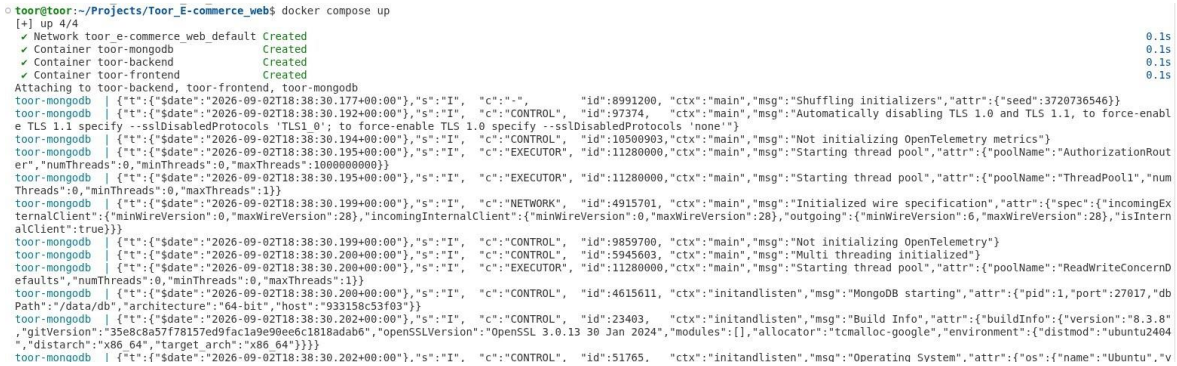
Inside a container, localhost refers to that container. A database running in another container should therefore be reached through the Docker network and the appropriate service/container name rather than assuming localhost means the database.

Why Compose helps

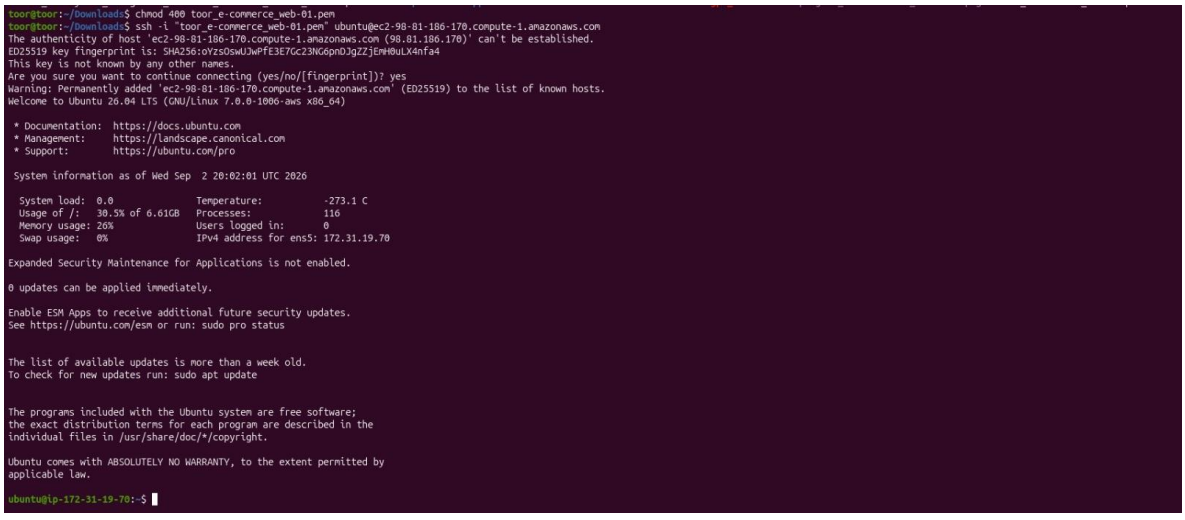
Docker Compose describes the multi-container application as one stack, so the frontend, backend, and database can be started, inspected, and stopped together.

Task 4: Troubleshooting Container Startup

Diagnosing the first Compose run



Database Startup / Error Evidence



Container Logs / Host Access

Q.2: How was the problem investigated?

Using Docker output instead of guessing

Read the Compose output

The startup output records which networks and containers were created and shows the point where the application or database failed to become healthy.

Inspect logs

Container logs provide the most direct evidence of what a process is doing. They can reveal connection failures, configuration mistakes, missing files, or a service that exited immediately.

Separate build problems from runtime problems

A successful image build does not guarantee a successful application startup. The screenshots illustrate why both stages must be checked: first confirm the image, then confirm the running service and its dependencies.

Practical lesson

Troubleshooting becomes faster when each layer is checked separately: image → container → network → dependency → application response.

Task 5: Docker Images & Registry

Verifying images before cloud deployment

Docker Image List

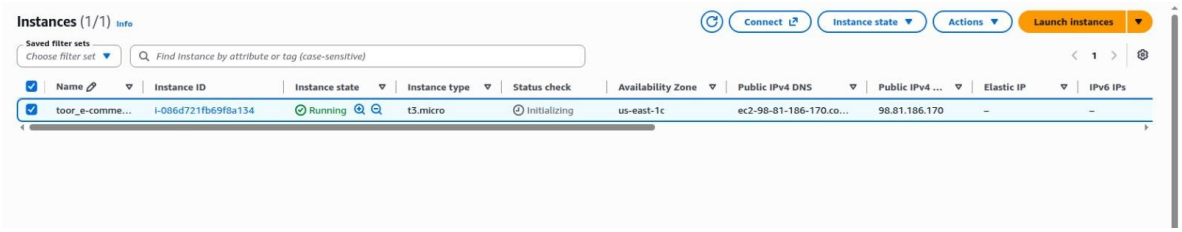
Image Build / Registry Evidence

Task 6: AWS EC2 Deployment

Preparing the cloud host

```
toor@toor:~/Projects/Toor_E-commerce_web$ docker run --rm -p 3000:80 abdullahtoor/toor_e-commerce_web:01
/docker-entrypoint.sh: /docker-entrypoint.d/ is not empty, will attempt to perform configuration
/docker-entrypoint.sh: Looking for shell scripts in /docker-entrypoint.d/
/docker-entrypoint.sh: Launching /docker-entrypoint.d/10-listen-on-ipv6-by-default.sh
10-listen-on-ipv6-by-default.sh: info: Getting the checksum of /etc/nginx/conf.d/default.conf
10-listen-on-ipv6-by-default.sh: info: Enabled listen on IPv6 in /etc/nginx/conf.d/default.conf
/docker-entrypoint.sh: Sourcing /docker-entrypoint.d/15-local-resolvers.envsh
/docker-entrypoint.sh: Launching /docker-entrypoint.d/20-envsubst-on-templates.sh
/docker-entrypoint.sh: Launching /docker-entrypoint.d/30-tune-worker-processes.sh
/docker-entrypoint.sh: Configuration complete; ready for start up
2026/09/02 19:21:15 [notice] 1#1: using the "epoll" event method
2026/09/02 19:21:15 [notice] 1#1: nginx/1.31.4
2026/09/02 19:21:15 [notice] 1#1: built by gcc 15.2.0 (Alpine 15.2.0)
2026/09/02 19:21:15 [notice] 1#1: OS: Linux 7.0.0-30-generic
2026/09/02 19:21:15 [notice] 1#1: getrlimit(RLIMIT_NOFILE): 1024:524288
2026/09/02 19:21:15 [notice] 1#1: start worker processes
2026/09/02 19:21:15 [notice] 1#1: start worker process 30
2026/09/02 19:21:15 [notice] 1#1: start worker process 31
2026/09/02 19:21:15 [notice] 1#1: start worker process 32
2026/09/02 19:21:15 [notice] 1#1: start worker process 33
2026/09/02 19:21:15 [notice] 1#1: start worker process 34
2026/09/02 19:21:15 [notice] 1#1: start worker process 35
172.17.0.1 - - [02/Sep/2026:19:21:26 +0000] "GET / HTTP/1.1" 200 3530 "-" "Mozilla/5.0 (X11; Linux x86_64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/151.0.0.0 Safari/537.36" "-"
172.17.0.1 - - [02/Sep/2026:19:21:26 +0000] "GET /css/style.css HTTP/1.1" 200 6451 "http://localhost:3000/" "Mozilla/5.0 (X11; Linux x86_64) AppleWebKit/537.36 (KHTML, like Gecko) Ch
rome/151.0.0.0 Safari/537.36" "-"
172.17.0.1 - - [02/Sep/2026:19:21:26 +0000] "GET /js/app.js HTTP/1.1" 200 2909 "http://localhost:3000/" "Mozilla/5.0 (X11; Linux x86_64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome
/151.0.0.0 Safari/537.36" "-"
172.17.0.1 - - [02/Sep/2026:19:21:26 +0000] "GET /js/api.js HTTP/1.1" 200 507 "http://localhost:3000/" "Mozilla/5.0 (X11; Linux x86_64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/
151.0.0.0 Safari/537.36" "-"
172.17.0.1 - - [02/Sep/2026:19:21:26 +0000] "GET /favicon.ico HTTP/1.1" 404 555 "http://localhost:3000/" "Mozilla/5.0 (X11; Linux x86_64) AppleWebKit/537.36 (KHTML, like Gecko) Chrom
e/151.0.0.0 Safari/537.36" "-"
2026/09/02 19:21:26 [error] 32#32: *2 open() "/usr/share/nginx/html/favicon.ico" failed (2: No such file or directory), client: 172.17.0.1, server: localhost, request: "GET /favicon.
ico HTTP/1.1", host: "localhost:3000", referrer: "http://localhost:3000/"
```

AWS EC2 Instance



Remote Host / SSH Session

Step 2: Pull and Run the Image

Moving the container workload from registry to EC2

Pull

The EC2 host retrieves the Docker image from the registry. This means the cloud machine does not need the original development project files just to start the already-built image.

Run

Docker creates a container from the pulled image. Port publishing connects a port on the EC2 host to the port used by the application inside the container.

Why this matters

The same container image built and verified earlier can now be used on the cloud host. This is the core reproducibility benefit of containerization.

Evidence

The deployment screenshots show the pull operation and the running container on the EC2 host.

Step 3: Verify the Running Container

Confirming the workload is actually serving

```
ubuntu@ip-172-31-19-70:~$ sudo docker push abdullahtoor/toor_e-commerce_web:01
The push refers to repository [docker.io/abdullahtoor/toor_e-commerce_web]
tag does not exist: abdullahtoor/toor_e-commerce_web:01
ubuntu@ip-172-31-19-70:~$ sudo docker pull abdullahtoor/toor_e-commerce_web:01
01: Pulling from abdullahtoor/toor_e-commerce_web
becf56fb25da: Pull complete
1ed8e39a7434: Pull complete
55afa1ecc21d: Pull complete
d94291c26261: Pull complete
0ea935727878: Pull complete
0d62d88506ba: Pull complete
8cdfe4f23778: Pull complete
da4dea1b00af: Pull complete
fb597529c916: Pull complete
44136fa355b3: Download complete
1c1228942627: Download complete
Digest: sha256:53b43fe3d92bd0573d0498a9ef4f8a643f0d863fcfbb9d3095ba6d046aa4fc01
Status: Downloaded newer image for abdullahtoor/toor_e-commerce_web:01
docker.io/abdullahtoor/toor_e-commerce_web:01
ubuntu@ip-172-31-19-70:~$ docker images
permission denied while trying to connect to the docker API at unix:///var/run/docker.sock
ubuntu@ip-172-31-19-70:~$ sudo docker images

IMAGE                                ID             DISK USAGE   CONTENT SIZE   EXTRA
abdullahtoor/toor_e-commerce_web:01   53b43fe3d92b       93.4MB         26.3MB
ubuntu@ip-172-31-19-70:~$
```

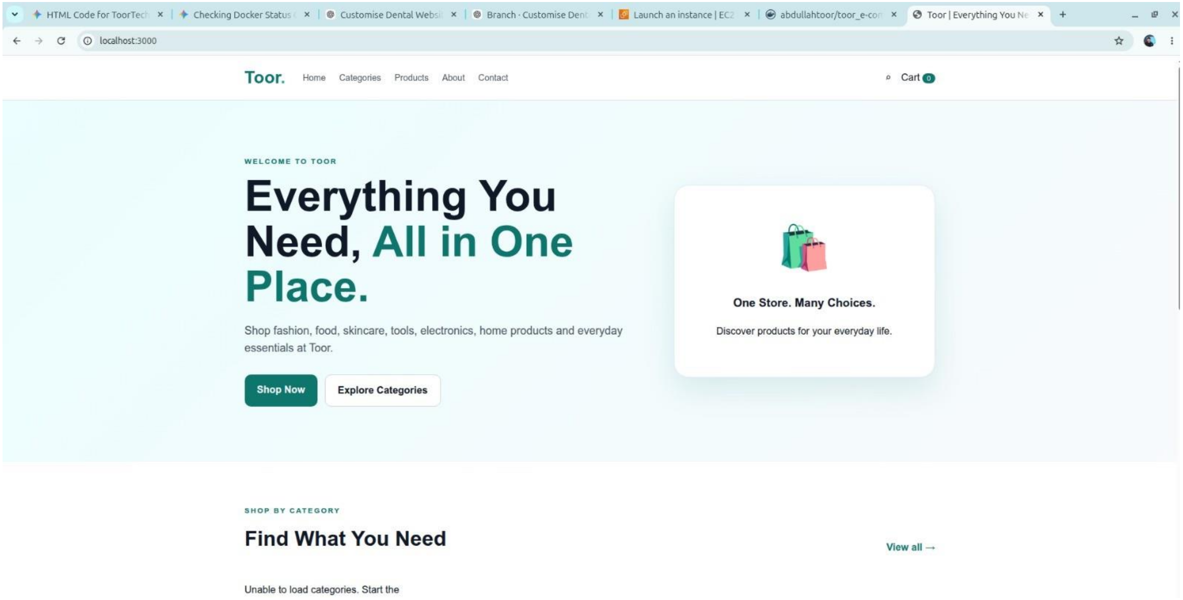
Pull Image on EC2

```
ubuntu@ip-172-31-19-70:~$ sudo docker run --rm -d -p 3000:3000 abdullahtoor/toor_e-commerce_web:01
436f672ab7afa9aa77e9c32a6e4155d9b25b06a451b1da35c1e994c8c0c5aa94
ubuntu@ip-172-31-19-70:~$ sudo docker ps
CONTAINER ID   IMAGE                                COMMAND                  CREATED          STATUS          PORTS                                                                                  NAMES
436f672ab7af   abdullahtoor/toor_e-commerce_web:01   "/docker-entrypoint.…"   24 minutes ago   Up 24 minutes   80/tcp, 0.0.0.0:3000->3000/tcp, [::]:3000->3000/tcp   bold_buck
ubuntu@ip-172-31-19-70:~$
```

Running Container

Deployment Architecture & Evidence

End-to-end view of the Toor deployment



Application Running on the EC2 Host

Docker Build / Verification Evidence

Lessons Learned

What this project demonstrated in practice

Containerization

I learned that a Dockerfile is a repeatable recipe for turning application source into an image, while a container is the running instance created from that image.

Networking

The project made container communication practical rather than theoretical. The frontend, backend, and database must be connected through an appropriate Docker network.

Troubleshooting

The first startup failure showed why logs and service status are essential. A deployment is not complete just because the image builds successfully.

Cloud deployment

AWS EC2 provides the remote compute host, while Docker provides the portable application runtime. Together they create a simple path from local development to cloud deployment.

Conclusion

From local application to Dockerized AWS deployment

Summary

This project provided a practical introduction to deploying a real application with Docker. I started with the Toor e-commerce application, built Docker images, brought multiple services together with Docker Compose, investigated a startup problem, verified the resulting images, and prepared an EC2 host for deployment.

Final result

The evidence demonstrates the complete deployment workflow: application → Docker image → container → Docker networking / Compose → registry → AWS EC2 → running application.

Next step

The next stage of the learning journey is to strengthen the deployment with environment variables, persistent volumes, health checks, CI/CD, and infrastructure automation while keeping the same clear troubleshooting mindset.

Key takeaway

Docker is not only a packaging tool; in this project it became the practical link between application development, repeatable deployment, and cloud infrastructure.